



크롤링 한 데이터로 통계를 보여주는 웹 사이트를 만들려고 하는데

데이터베이스에 데이터 종류 별로 마지막 크롤 시점을 저장하는 테이블을 생성하는 부분에 대해 어떻게 생각해?

네, 좋은 방향입니다. 다만 데이터 종류별 마지막 크롤 시점만 단일 컬럼으로 저장하는 것에서 끝내기보다는, “현재 상태용 테이블 1개 + 실행 이력 테이블 1개”로 나누는 설계를 추천합니다. 증분 수집에서는 보통 마지막 처리 지점인 워터마크를 별도 메타데이터로 관리하는 패턴이 쓰이고, 단순 “마지막 실행 시각”만 저장하면 실패 재시도나 부분 성공 같은 운영 이슈를 다루기 어렵습니다.^{[1] [2] [3]}

왜 필요한가

통계를 보여주는 사이트라면 사용자는 “이 데이터가 언제까지 반영됐는지”를 보게 되고, 운영 측면에서는 크롤러가 마지막으로 언제 성공했는지와 현재 기준 시점이 무엇인지가 매우 중요합니다. AWS Glue 도 크롤러별로 마지막 실행 시간, 시작/종료 시간, 상태, 변경 수 같은 실행 정보를 따로 보여주며, 이런 운영 메타데이터는 실제 서비스에서 꽤 유용합니다.^[3]

또한 증분 수집은 보통 high watermark 방식으로 마지막 처리 기준값을 저장해 다음 배치에서 그 이후만 읽습니다. 즉 “마지막 크롤 시각”은 UI 표시용으로는 좋지만, 실제 재수집 기준으로는 `last_success_watermark` 같은 별도 필드가 더 중요합니다.^{[2] [1]}

추천 구조

실무적으로는 아래처럼 나누면 깔끔합니다. 메타데이터 설계에서도 `last_crawled_at` 같은 현재 상태 필드와 별도 실행 흐름을 분리하는 접근이 자주 보입니다.^[4]

1) 현재 상태 테이블

예: `crawl_target_status`

- `target_type` 또는 `data_type` — 어떤 종류의 데이터를 의미하는지, unique 처리.
- `last_started_at` — 마지막 실행 시작 시각.
- `last_finished_at` — 마지막 실행 종료 시각.
- `last_succeeded_at` — 마지막 성공 시각.
- `last_failed_at` — 마지막 실패 시각.
- `last_watermark` — 다음 증분 수집의 기준값.
- `status` — `idle`, `running`, `success`, `failed`.
- `row_count_last_run` — 직전 실행에서 수집한 건수.

- `updated_at` — 메타데이터 갱신 시각.

이 테이블은 “지금 이 데이터셋 상태가 어떤가”를 빠르게 조회하는 용도입니다. 대시보드에서 “주가 데이터: 2026-05-16 19:55 기준 반영”처럼 보여주기 좋습니다.^{[2] [3]}

2) 실행 이력 테이블

예: `crawl_run_history`

- `id`
- `target_type`
- `started_at`
- `finished_at`
- `status`
- `watermark_from`
- `watermark_to`
- `fetched_count`
- `inserted_count`
- `updated_count`
- `error_message`
- `trigger_type` — `schedule`, `manual`, `retry`

이력 테이블은 장애 분석과 재처리에 필요합니다. 단순히 마지막 시각만 남기면 “왜 데이터가 비었는지”, “어느 시점부터 누락됐는지”, “실패 후 재시도했는지”를 추적하기 어렵습니다.^[3]

단일 테이블만 둘 때의 한계

`data_type`, `last_crawled_at` 정도만 두는 구조는 시작하기 쉽지만, 아래 상황에서 금방 부족해집니다. 워터마크 테이블이 단순하고 가볍다는 장점은 있지만, 변경 추적 요구가 커질수록 추가 정보가 필요하다는 점도 같이 고려해야 합니다.^{[5] [2]}

- 실행은 시작됐지만 아직 끝나지 않음.
- 크롤은 실패했는데 `last_crawled_at`만 갱신돼 버림.
- 일부 페이지만 성공한 부분 성공 상태.
- “마지막 성공 시점”과 “마지막 시도 시점”을 구분해야 함.
- 특정 날짜 범위 재수집이 필요함.

특히 `last_crawled_at`를 “시도 시각”으로 볼지 “성공 시각”으로 볼지 정의가 모호하면 버그가 생기기 쉽습니다. 그래서 최소한 `last_started_at`과 `last_succeeded_at`은 분리하는 편이 안전합니다.^{[2] [3]}

구현 팁

크롤링 대상이 명확히 나뉜다면 `data_type`을 enum처럼 관리해도 되지만, 소스가 늘어날 가능성이 있으면 `source`, `entity_type`, `frequency`까지 분리해두는 편이 확장에 유리합니다. 메타데이터 카탈로그 설계에서도 자산 유형, 소스 식별자, 마지막 크롤 시점 같은 구조화된 필드를 별도로 두는 방식이 확장성과 조화성에 유리하다고 설명합니다.^[4]

예를 들면 이런 식입니다.

```
create table crawl_target_status (
  target_type varchar(50) primary key,
  status varchar(20) not null,
  last_started_at datetime null,
  last_finished_at datetime null,
  last_succeeded_at datetime null,
  last_failed_at datetime null,
  last_watermark varchar(100) null,
  row_count_last_run int not null default 0,
  updated_at datetime not null
);
```

그리고 이력 테이블은 `append-only`로 두는 편이 좋습니다. AWS Glue처럼 실행 단위의 시작, 종료, 상태, 변경량을 남겨두면 운영성이 좋아집니다.^[3]

권장 판단

당장 소규모 프로젝트라도 “데이터 종류별 마지막 크롤 시점 저장 테이블”은 두는 게 맞습니다. 다만 그 테이블을 단순 표시용으로 보지 말고, 성공 시점, 실행 상태, 워터마크를 함께 가진 운영 메타데이터 테이블로 설계하면 나중에 훨씬 덜 고생합니다.^{[1][2][3]}

개인적으로는 다음 기준을 추천합니다.

- 작게 시작: `crawl_target_status` 1개.
- 운영 안정성 필요: `crawl_run_history` 추가.
- 증분 수집 있음: `last_watermark` 필수.
- 사용자 화면에 노출: `last_succeeded_at`를 기준 시각으로 사용.

원하면 다음 답변에서 MySQL 기준으로 테이블 DDL을 더 구체적으로 짜드릴게요.

✻

1. <https://www.sciencedirect.com/science/article/pii/S2214579625000693>
2. <https://techcommunity.microsoft.com/blog/fasttrackforazureblog/robust-data-ingestion-with-high-water-marking/3707480>
3. <https://docs.aws.amazon.com/glue/latest/dg/console-crawlers-details.html>
4. <https://www.techinterview.org/post/3233470072/1ld-metadata-catalog/>
5. https://www.linkedin.com/posts/akilan-k-t-546ab31b5_watermark-vs-change-data-feed-cdf-in-databricks-activity-7383939748995854337-oz4c

6. <https://discuss.google.dev/t/last-access-timestamp/148058>
7. <https://www.snowflake.com/en/developers/guides/create-an-llm-generated-data-catalog-using-data-crawler/>
8. <https://jazz.net/forum/questions/102485/where-in-the-data-warehouse-is-the-timestamp-for-the-last-data-collection-or-data-movement-job>
9. <https://github.com/petersonjr/MetadataCrawler>
10. <https://github.com/ClickHouse/ClickHouse/issues/1582>
11. https://www.youtube.com/watch?v=tW9Q_1W8xZo
12. <https://stackoverflow.com/questions/3241033/designing-database-to-hold-different-metadata-information>
13. <https://docs.aws.amazon.com/glue/latest/dg/add-crawler.html>
14. <https://www.pracdata.io/p/metadata-layer-design-of-open-table-formats>
15. <https://dev.to/alexmercedcoder/the-metadata-structure-of-modern-table-formats-i81>